



Started on	Tuesday, 25 August 2026, 2:32 PM
State	Finished
Completed on	Tuesday, 25 August 2026, 2:36 PM
Time taken	3 mins 58 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct Mark 1.00 out of 1.00**Problem Statement**

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int n;
6      scanf("%d", &n);
7
8      int a[n];
9
10     for (int i = 0; i < n; i++)
11         scanf("%d", &a[i]);
12
13     int low = 0;
14     int high = n - 1;
15     int firstZero = n;
16
17     while (low <= high)
18     {
19         int mid = (low + high) / 2;
20
21         if (a[mid] == 0)
22         {
23             firstZero = mid;
24             high = mid - 1;
25         }
26         else
27         {
28             low = mid + 1;
29         }
30     }
31
32     printf("%d", n - firstZero);
33
34     return 0;
35 }
```

	Input	Expected	Got	
✓	5 1 1 1 0 0	2	2	✓

	Input	Expected	Got	
✓	10 1 1 1 1 1 1 1 1 1 1 1 1	0	0	✓
✓	8 0 0 0 0 0 0 0 0 0 0	8	8	✓
✓	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)



Started on	Thursday, 27 August 2026, 11:19 AM
State	Finished
Completed on	Thursday, 27 August 2026, 11:24 AM
Time taken	5 mins 4 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`

Output: `3`

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: `2`

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-231 <= nums[i] <= 231 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```
1  #include<stdio.h>
2  int main()
3  {
4      int n;
5      scanf("%d",&n);
6      int arr[n];
7      for(int i=0;i<n;i++)
8      {
9          scanf("%d",&arr[i]);
10     }
11     for(int i=0;i<n;i++)
12     {
13         int count=0;
14         for(int j=0;j<n;j++)
15         {
16             if(arr[i]==arr[j])
17             {
18                 count++;
19             }
20         }
21         if(count>n/2)
22         {
23             printf("%d",arr[i]);
24             break;
25         }
26     }
```

```
28 | }
```

	Input	Expected	Got	
✓	3	3	3	✓
	3 2 3			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)





Started on	Monday, 31 August 2026, 6:33 PM
State	Finished
Completed on	Monday, 31 August 2026, 6:36 PM
Time taken	3 mins 44 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct Mark 1.00 out of 1.00**Problem Statement:**

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int n, x;
6      scanf("%d", &n);
7
8      int arr[n];
9
10     for (int i = 0; i < n; i++)
11     {
12         scanf("%d", &arr[i]);
13     }
14
15     scanf("%d", &x);
16
17     int low = 0;
18     int high = n - 1;
19     int floor = -1;
20
21     while (low <= high)
22     {
23         int mid = low + (high - low) / 2;
24
25         if (arr[mid] == x)
26         {
27             floor = arr[mid];
28             break;
29         }
30         else if (arr[mid] < x)
31         {
32             floor = arr[mid];
33             low = mid + 1;
34         }
35         else
36         {
37             high = mid - 1;
38         }
39     }
40
41     printf("%d\n", floor);
42
43     return 0;
44 }
```

	Input	Expected	Got	
✓	6	2	2	✓
	1			
	2			
	8			
	10			
	12			
	19			
	5			
✓	5	85	85	✓
	10			
	22			
	85			
	108			
	129			
	100			
✓	7	9	9	✓
	3			
	5			
	7			
	9			
	11			
	13			
	15			
	10			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)



Started on	Monday, 31 August 2026, 6:37 PM
State	Finished
Completed on	Monday, 31 August 2026, 6:42 PM
Time taken	5 mins 9 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct Mark 1.00 out of 1.00**Problem Statement:**

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int binarySearch(int arr[], int low, int high, int key)
4  {
5      if (low > high)
6          return -1;
7
8      int mid = (low + high) / 2;
9
10     if (arr[mid] == key)
11         return mid;
12
13     if (arr[mid] < key)
14         return binarySearch(arr, mid + 1, high, key);
15
16     return binarySearch(arr, low, mid - 1, key);
17 }
18
19 int findPair(int arr[], int n, int x, int i)
20 {
21     if (i >= n - 1)
22         return 0;
23
24     int required = x - arr[i];
25
26     int pos = binarySearch(arr, i + 1, n - 1, required);
27
28     if (pos != -1)
29     {
30         printf("%d\n", arr[i]);
31         printf("%d\n", arr[pos]);
32         return 1;
33     }
34
35     return findPair(arr, n, x, i + 1);
36 }
37
38 int main()
39 {
40     int n, x;
41
42     scanf("%d", &n);
43
44     int arr[n];
45
46     for (int i = 0; i < n; i++)
47     {
48         scanf("%d", &arr[i]);
49     }
50

```

51
52

scanf("%d", &x);

	Input	Expected	Got	
✓	4	4	4	✓
	2	10	10	
	4			
	8			
	10			
	14			
✓	5	No	No	✓
	2			
	4			
	6			
	8			
	10			
	100			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)



Started on	Monday, 31 August 2026, 6:42 PM
State	Finished
Completed on	Monday, 31 August 2026, 6:52 PM
Time taken	9 mins 38 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```

1  #include <stdio.h>
2
3  void quickSort(int arr[], int low, int high)
4  {
5      if (low < high)
6      {
7          int pivot = arr[high];
8          int i = low - 1;
9
10         for (int j = low; j < high; j++)
11         {
12             if (arr[j] < pivot)
13             {
14                 i++;
15
16                 int temp = arr[i];
17                 arr[i] = arr[j];
18                 arr[j] = temp;
19             }
20         }
21
22         int temp = arr[i + 1];
23         arr[i + 1] = arr[high];
24         arr[high] = temp;
25
26         int p = i + 1;
27
28         quickSort(arr, low, p - 1);
29         quickSort(arr, p + 1, high);
30     }
31 }
32
33 int main()
34 {
35     int n;
36     scanf("%d", &n);
37
38     int arr[n];
39
40     for (int i = 0; i < n; i++)
41     {
42         scanf("%d", &arr[i]);
43     }
44
45     quickSort(arr, 0, n - 1);
46

```

```
47 |     for (int i = 0; i < n; i++)
48 |     {
49 |         printf("%d ", arr[i]);
50 |     }
51 |
52 |     return 0;
```

	Input	Expected	Got	
✓	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)